



Rapport de vérification

Correctifs sécurité client

Easy Dunya

Date du rapport : 12/08/2026

Site : <https://easydunya.netlify.app>

Application : APK Android (Capacitor) + PWA Web

Périmètre : 4 correctifs client — audit expert

STATUT GLOBAL : TOUS LES TESTS VALIDÉS

Ce document résume la mise en place et la vérification des quatre correctifs de sécurité identifiés lors de l'audit expert du client Easy Dunya. Les tests ont été exécutés manuellement sur le navigateur Chrome et sur l'APK Android installé sur téléphone.

Sommaire

1. Protection open redirect (notifications push)
2. En-têtes de sécurité HTTP (netlify.toml)
3. Redirection après connexion validée (Login.tsx)
4. Codes de confirmation imprévisibles (codes.ts)

1. Protection contre l'open redirect (notifications)

Problème identifié

Un attaquant pouvait forger une notification push contenant une URL externe (ex. <https://evil.com>). Au clic, l'utilisateur était redirigé vers un site malveillant imitant Easy Dunya (phishing).

Correctif appliqué

- Web / PWA : fonction `sameOriginUrl()` dans `public/push-sw.js` — seules les URLs de la même origine (`easydunya.netlify.app`) sont autorisées.
- APK Android : fonction `safeAppPath()` dans `src/lib/nativePush.ts` — même règle au clic sur une notification FCM native.
- Architecture : `sw.js` charge `push-sw.js` via `importScripts()` ; `sameOriginUrl` n'apparaît pas dans `sw.js` mais dans `push-sw.js`.

Tests effectués — Web (navigateur Chrome)

- Vérification du fichier `push-sw.js` en production : présence de `sameOriginUrl` confirmée.
- Test console (connecté admin + non connecté) : `showNotification` avec `data.url = "https://evil.com/faux-easydunya"`.
Résultat : au clic, redirection vers Easy Dunya uniquement, jamais vers `evil.com`.
- Service Worker actif (`sw.js`) vérifié dans DevTools → Application → Service workers.

Tests effectués — APK Android

- Insertion SQL d'une notification test avec URL malveillante dans `data.url`.
- Notification reçue ; clic → ouverture de l'APK Easy Dunya (pas de navigateur externe, pas de redirection vers `evil.com`).
- Notification visible dans la cloche in-app — comportement normal (même entrée en base).

[OK] Point 1 — VALIDÉ : protection open redirect active sur Web et APK.

2. En-têtes de sécurité HTTP (netlify.toml)

Problème identifié

Le site était servi sans en-têtes de sécurité standards, exposant à : clickjacking (iframe invisible), devinette de type MIME (vecteur XSS), fuite d'URL via Referer vers des sites tiers, et absence de forçage HTTPS au niveau navigateur (HSTS).

Correctif appliqué (netlify.toml)

- Strict-Transport-Security : max-age=31536000; includeSubDomains
- X-Frame-Options : DENY
- Content-Security-Policy : frame-ancestors 'none'
- X-Content-Type-Options : nosniff
- Referrer-Policy : strict-origin-when-cross-origin
- Permissions-Policy : camera=(), microphone=(), payment=(), geolocation=(self)

Tests effectués

- Méthode 1 — Chrome DevTools (Réseau → Response Headers) : les 6 en-têtes présents et conformes.
- Méthode 2 — Outil en ligne securityheaders.com : note satisfaisante (A / B+).
- Méthode 3 — Test clickjacking (iframe locale) : refus d'affichage dans l'iframe — protection active.
- Méthode 4 — PowerShell Invoke-WebRequest -Method Head : en-têtes confirmés côté serveur Netlify Edge.

[OK] Point 2 — VALIDÉ : en-têtes de sécurité HTTP déployés et actifs en production.

3. Redirection après connexion validée (Login.tsx)

Problème identifié

Après connexion, l'application naviguait vers `state.from` sans validation. Le risque d'exploitation directe était faible (`state` posé par notre code), mais ce chemin devient exploitable avec des contournements par backslash (`//host`, `^host`) comme dans les CVE récentes de React Router.

Correctif appliqué (src/pages/Login.tsx)

- Validation par regex : `/^\/(?![/\])` — seuls les chemins internes commençant par `/` (sans `//` ni `^`) sont acceptés.
- Toute valeur invalide est remplacée par `/` (accueil).
- Cas légitime conservé : redirection vers `/admin`, `/profile`, `/trips/...` après connexion depuis une page protégée.

Tests effectués

- Test A — Accès `/admin` sans connexion → login → redirection vers `/admin` (OK).
- Test B — Injection `state.from = "//evil.com"` via `history.replaceState` → après login, reste sur `easydunya.netlify.app` (accueil `/`).
- Test C — Injection `state.from = "^\\evil.com"` (backslash) → bloqué, redirection vers `/`.
- Test D — Injection `state.from = "https://evil.com/phishing"` → bloqué, redirection vers `/`.
- Test E — Login depuis page voyage (`/trips/...`) → retour sur la page du voyage.

[OK] Point 3 — VALIDÉ : aucune redirection externe après connexion ; chemins internes légitimes fonctionnels.

4. Codes de confirmation imprévisibles (codes.ts)

Problème identifié

Le code à 6 caractères d'une réservation est une clé d'accès : quiconque le connaît voit le nom et le téléphone du passager (page « Vérifier une réservation »). L'ancien générateur utilisait `Math.random()` (prévisible) avec un léger biais statistique via le modulo.

Correctif appliqué (src/lib/codes.ts)

- Génération exclusive via `crypto.getRandomValues()` — jamais `Math.random()`.
- Rejet des valeurs biaisées (rejection sampling) pour une distribution uniforme sur l'alphabet de 31 caractères.
- Alphabet sans caractères ambigus : pas de 0/O, 1/I/L — format 6 caractères majuscules.
- Espace des codes : $31^6 \approx 887$ millions de combinaisons.

Tests effectués

- Test A — Création de réservation : code 6 caractères, alphabet conforme, codes distincts entre deux réservations.
- Test B — Page /check : codes inventés (AAAAAA, 000000, OIL111...) → réservation introuvable ou refusée.
- Test C — Inspection du build : `getRandomValues` présent, `Math.random` absent dans la logique de génération.
- Test D — Génération console (10 codes) : tous valides, variés, sans caractères interdits.
- Test E — Tentatives de devinette (codes aléatoires) : aucun code inventé ne donne accès aux données passager.

[OK] Point 4 — VALIDÉ : codes cryptographiquement aléatoires, non prévisibles.



. Synthèse et conclusion

Les quatre correctifs de sécurité client identifiés lors de l'audit expert ont été implémentés dans le dépôt GitHub (branche main), déployés sur Netlify (<https://easydunya.netlify.app>) et vérifiés manuellement par l'équipe Easy Dunya sur navigateur Chrome et APK Android.

Tableau récapitulatif

N°	Correctif	Fichier source	Statut
1	Open redirect notifications	push-sw.js, nativePush.ts	OK — Web + APK
2	En-têtes HTTP sécurité	netlify.toml	OK — 4 méthodes
3	Redirection login validée	Login.tsx	OK — 5 tests
4	Codes confirmation sécurisés	codes.ts	OK — 5 tests

CONCLUSION : Les 4 points de l'audit sécurité client sont corrigés, déployés en production et validés par tests manuels.

Document généré automatiquement — Easy Dunya / mounaStart/easydunya. Régénération : python scripts/generate_rapport_securite_pdf.py